

Introducción a la Programación con Python

Semana 3: Excepciones y Manejo de Errores

∇Nablabs

Contenido

- 1 ¿Qué son las excepciones?
- 2 Tipos de errores comunes
- 3 Bloques try y except
- 4 Cláusula else
- 5 Cláusula finally
- 6 Creación de excepciones con raise
- 7 Buenas prácticas
- 8 Proyecto: Validador robusto

¿Qué es una excepción?



Evento que interrumpe el flujo normal del programa

Permite manejar errores sin que el programa se detenga



Python levanta excepciones cuando encuentra problemas



`try, except, finally`

Error de sintaxis

Se produce cuando el código no sigue las reglas de Python

```
1 # Error: falta cerrar el paréntesis
2 print("Hola mundo"
```

Error

SyntaxError: '(' was never closed

 Los SyntaxError deben corregirse antes de ejecutar

ValueError !

Error de valor

Ocurre cuando una función recibe un argumento del tipo correcto pero valor inapropiado

```
1 numero = int("abc")
```

Error

ValueError: invalid literal for int() with base 10: 'abc'

 Común al convertir strings a números

NameError ?

Error de nombre

Se produce cuando se intenta usar una variable que no existe

```
1 print(variable_no_definida)
```

! Error

NameError: name 'variable_no_definida' is not defined

i Verifica que hayas definido todas tus variables

Try-Except básico

¿Cómo funciona?

El bloque try intenta ejecutar el código, except captura el error

```
1 try:
2     numero = int(input("Ingresa un número: "))
3     print(f"El número es: {numero}")
4 except ValueError:
5     print("Error: debes ingresar un número válido")
```

➡ Salida (entrada: 'abc')

Error: debes ingresar un número válido

✓ El programa continúa ejecutándose sin fallar

Múltiples excepciones

```
1 try:
2     lista = [1, 2, 3]
3     indice = int(input("Índice: "))
4     print(lista[indice])
5 except ValueError:
6     print("Error: debes ingresar un número")
7 except IndexError:
8     print("Error: índice fuera de rango")
```

↓ Entrada: 'abc'

Error: debes ingresar un
número

↓ Entrada: 10

Error: índice fuera de rango

¿Para qué sirve?

Se ejecuta solo si NO ocurre ninguna excepción

```
1 try:
2     numero = int(input("Ingresa un número: "))
3 except ValueError:
4     print("Error: no es un número válido")
5 else:
6     print(f"Perfecto! El número es: {numero}")
```

➡ Salida (entrada: '42')

Perfecto! El número es: 42

💡 Útil para separar código que debe ejecutarse solo si no hay errores

Cláusula finally

¿Para qué sirve?

Se ejecuta SIEMPRE, haya o no haya error

```
1 try:
2     archivo = open("datos.txt", "r")
3     contenido = archivo.read()
4 except FileNotFoundError:
5     print("Error: archivo no encontrado")
6 finally:
7     print("Operación terminada")
8     # archivo.close() si se abrió
```

➔ Salida

```
Error: archivo no encontrado
Operación terminada
```

i Ideal para liberar recursos (cerrar archivos, conexiones, etc.)

Lanzar excepciones con raise

¿Cómo crear excepciones?

Puedes lanzar excepciones manualmente con raise

```
1 def verificar_edad(edad):
2     if edad < 0:
3         raise ValueError("La edad no puede ser negativa")
4     if edad < 18:
5         raise Exception("Debes ser mayor de edad")
6     return "Acceso permitido"
7
8 try:
9     print(verificar_edad(-5))
10 except ValueError as e:
11     print(f"Error de valor: {e}")
```

➔ Salida

Error de valor: La edad no puede ser negativa

Recomendaciones

- ✓ Captura excepciones específicas, no todas genéricas
- ✓ No dejes bloques except vacíos
- ✓ Usa mensajes de error descriptivos
- ✓ Registra los errores para debugging
- ✓ No uses excepciones para control de flujo normal
- ✓ Limpia recursos en finally o usa context managers

Ejemplo: ¿Qué evitar? ❌

❌ Malo:

```
1 try:
2     # código
3     pass
4 except:
5     pass
```

✅ Bueno:

```
1 try:
2     # código
3     pass
4 except ValueError as e:
5     print(f"Error: {e}")
```

⚠ Capturar todas las excepciones oculta errores importantes

```
1 def validar_usuario():
2     while True:
3         try:
4             nombre = input("Nombre: ")
5             if not nombre.strip():
6                 raise ValueError("El nombre no puede
7 estar vacío")
8
9             edad = int(input("Edad: "))
10            if edad < 0 or edad > 120:
11                raise ValueError("Edad debe estar entre 0
12 y 120")
13
14            return {"nombre": nombre, "edad": edad}
15
16        except ValueError as e:
17            print(f"Error: {e}. Intenta de nuevo.")
18        except KeyboardInterrupt:
19            print("\nOperación cancelada")
20            return None
```

```
1 try:
2     datos = validar_usuario()
3     if datos:
4         print(f"\nUsuario registrado:")
5         print(f"    Nombre: {datos['nombre']}")
6         print(f"    Edad: {datos['edad']}")
7 except Exception as e:
8     print(f"Error inesperado: {e}")
9 finally:
10    print("\nGracias por usar el validador")
```

➡ Ejemplo de salida

```
Nombre: Juan
Edad: 25
Usuario registrado:
Nombre: Juan
Edad: 25
Gracias por usar el validador
```

Más excepciones útiles

- ✚ **TypeError**: Operación en tipo incorrecto
- ✚ **ZeroDivisionError**: División por cero
- ✚ **KeyError**: Clave no existe en diccionario
- ✚ **FileNotFoundError**: Archivo no encontrado
- ✚ **ImportError**: Módulo no puede importarse

```
1 try:
2     resultado = 10 / 0
3 except ZeroDivisionError:
4     print("No se puede dividir por cero")
```


¿Qué aprendimos hoy?

- ✓ Tipos de errores comunes (SyntaxError, ValueError, NameError)
- ✓ Bloques try y except
- ✓ Cláusula else
- ✓ Cláusula finally
- ✓ Lanzar excepciones con raise
- ✓ Buenas prácticas en manejo de errores
- ✓ Proyecto: Validador robusto de datos

¡Escribe código robusto!



¡Preguntas!

Gracias por su atención